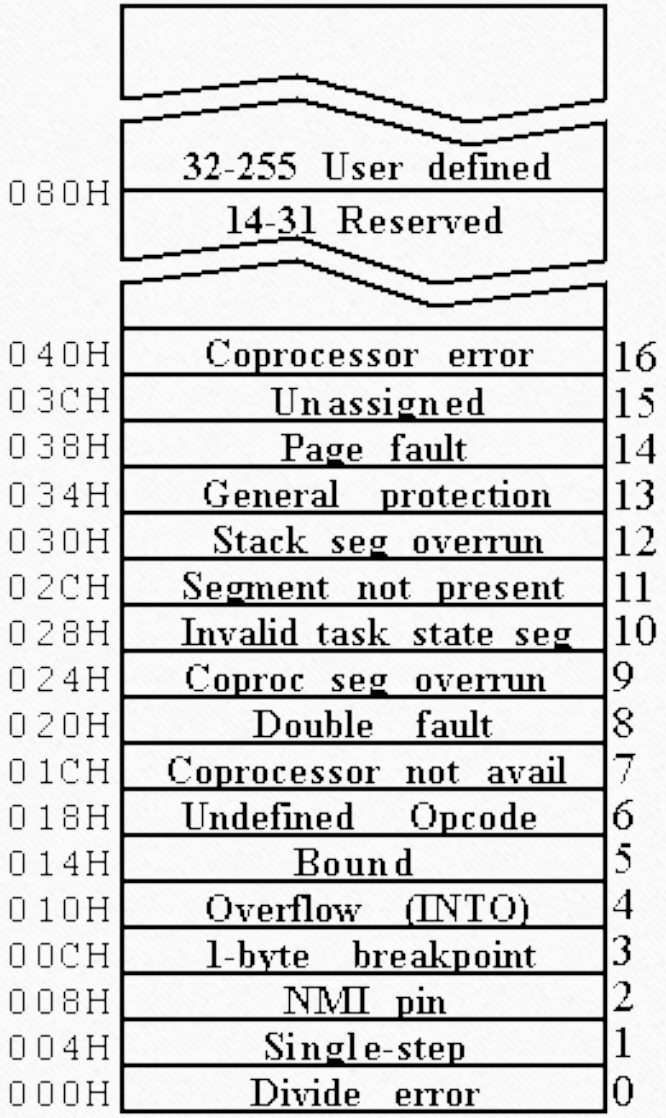
Lecture 9

COAL

Interrupt Vector Table (IVT)

- The IVT is a table of addresses the CPU uses to locate the handler for different types of interrupts and exceptions.
- The processor uses the interrupt number to index the Interrupt Vector Table (IVT) to get the address of a special type of procedure called an Interrupt Service Routine (ISR).
- There are 256 entries in IVT.
- IVT resides in memory at physical address 0x00000.
- Each entry stores the logical address, which is 4 bytes; the total size of the IVT is 256 x 4B = 1 KB
- Total Size is 1024B



The interrupt vector table is located in the first 1024 bytes of memory at addresses 000000H through 0003FFH.

There are 256 4-byte entries (segment and offset in real mode).

Seg high	Seg low	Offset high	Offset low
Byte 3	Byte 2	Byte 1	Byte 0

Exception Handling

- Unexpected events disrupting program execution (e.g., divide-by-zero, invalid memory access).
- Common Exceptions: Divide Error, Page Fault, General Protection Fault
- Handling Steps:
  - Detection: CPU or program identifies the exception.
  - IVT Lookup: Determines the handler.
  - Execution: Handler resolves or logs the issue.
  - Resolution: Program continues or terminates.

Interrupt vs Exception

Feature	Interrupt	Exception
Definition	An external event that signals the CPU to handle it.	An internal event caused by the program itself.
Source	External devices like keyboard, timer, or disk.	CPU or software issues like divide-by-zero or invalid memory access.
Examples	- Keyboard press - Timer reaching zero	- Division by zero - Accessing invalid memory
Trigger Type	Asynchronous (can happen anytime).	Synchronous (happens during program execution).
Handling	CPU jumps to an **interrupt service routine (ISR)**.	CPU jumps to an **exception handler**.
Recoverability	Often recoverable (e.g., handling keypresses).	Can be recoverable or may terminate the program.

Hooking

- The process of modifying or replacing an existing entry in the Interrupt Vector Table to redirect the handling of a specific interrupt to a custom Interrupt Service Routine (ISR).
- This technique is commonly used in low-level programming to intercept or extend the functionality of system interrupts.

Purpose of Hooking

- Customizing system behavior.
- Adding new functionality or debugging.
- Creating hooks for software or hardware interrupts.

Process of Hooking

- Locate the IVT entry for the desired interrupt (e.g., using its interrupt number).
- Save the original ISR address (optional, for restoration).
- Replace the IVT entry with the address of the custom ISR.

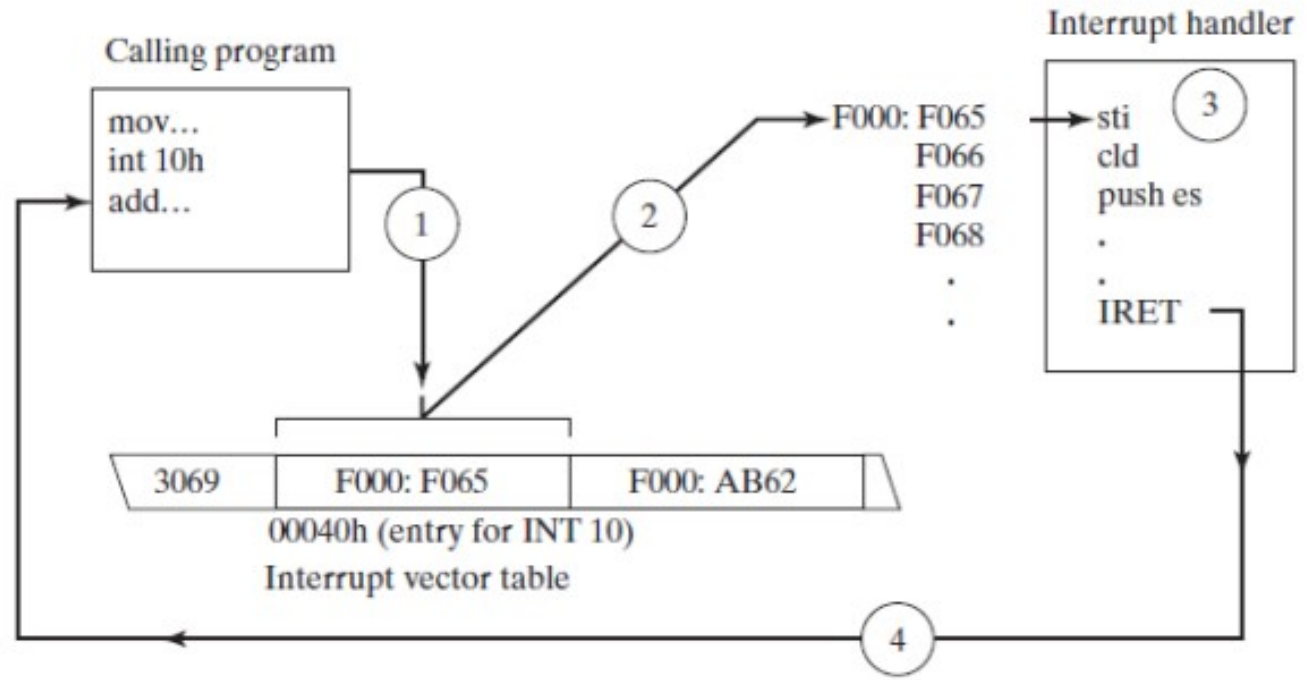
Risks and Applications

- **Risks**:
  - Incorrect modifications can crash the system.
  - Can cause conflicts if multiple programs attempt to hook the same interrupt.
- **Applications**:
  - Debugging tools.
  - Enhancing keyboard or mouse input.
  - Implementing custom OS-level features.

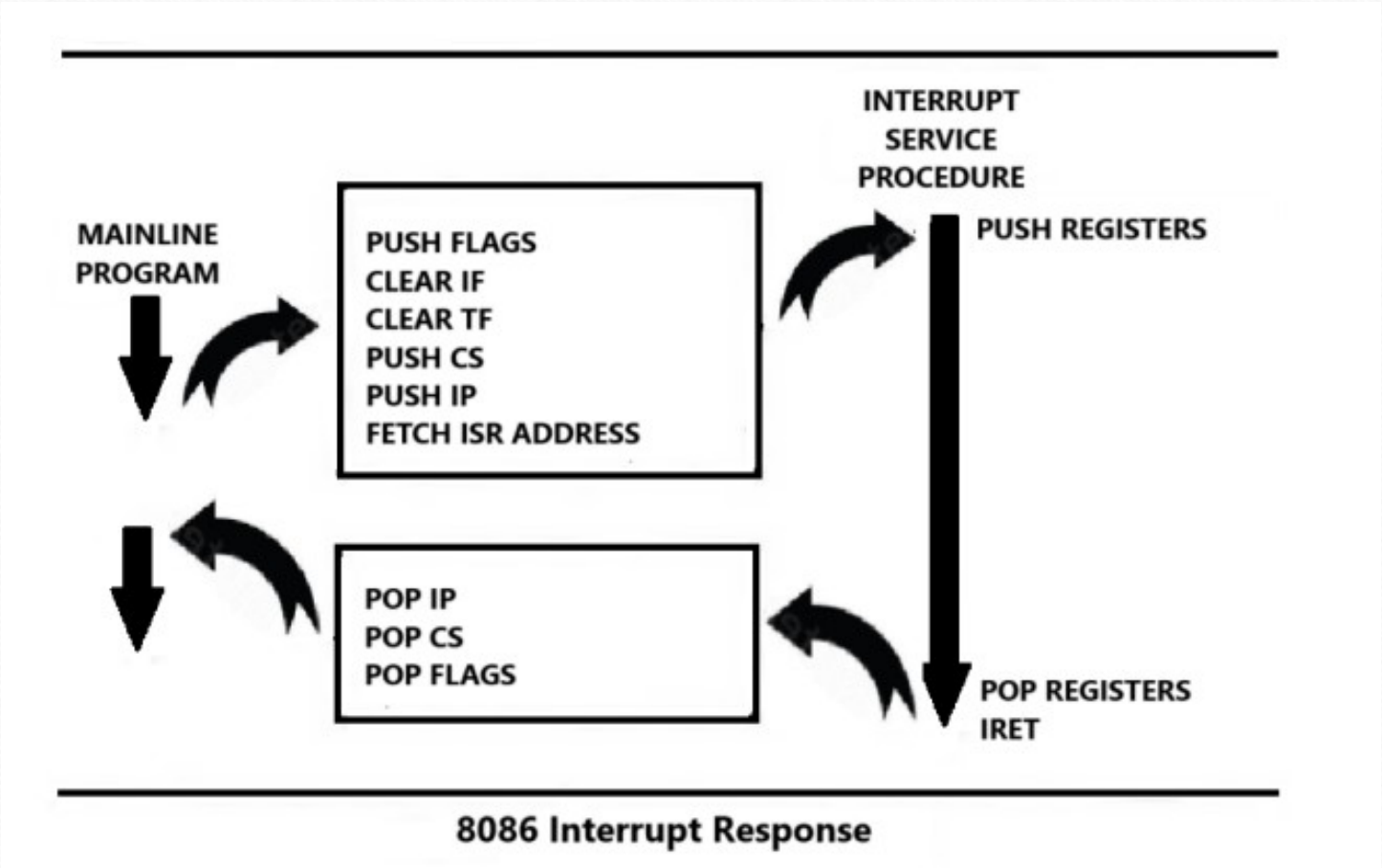
IRET Instruction

- Used to return from an interrupt service routine (ISR) to the interrupted program.
- Pops the flags, CS and IP registers from the stack.
- Restores the processor state to what it was before the interrupt occurred.

The IRET instruction also pops the flag register from the stack, whereas the RET instruction does not



Interrupt Vectoring Process



8086 Interrupt Response

Step by Step Execution of ISR

1. It decrements stack pointer by 2 and pushes the flag register on the stack.
2. It disables the INTR interrupt input by clearing the interrupt flag in the flag.
3. It resets the trap flag in the flag register.
4. It decrements stack pointer by 2 and pushes the current code segment register contents on the stack.
5. It decrements stack pointer by 2 and pushes the current instruction pointer contents on the stack.
6. It does an indirect jump at the start of the procedure by loading the CS and IP values for the start of the interrupt service routine (ISR).
7. An IRET instruction at the end of the interrupt service procedure returns execution to the main program.
8. The ISR should push registers to stack before using them and pop them before IRET.

Hook INT#0h

```
.model small
.data
.code
mov ax,@data
mov ds,ax
mov ax,0
mov es,ax
mov word ptr es:[0 * 4 + 0],isr0 ; offset address of ISR0
mov word ptr es:[ 0 * 4 + 2],cs ; Segment address of ISR0
mov ax,100
mov bl,0
div bl ;Dividing a number by zero to check if the hooking
.exit
ISR0 proc
jmp l@1
msg: db "Divide by Zero Exception!$"
l@1:
mov dx,offset msg
mov ah,9
int 0x21

IRET
ISR0 endp
```

Hook INT#4h

```
.model small
.data
.code
mov ax,@data
mov ds,ax
mov ax,0
mov es,ax
mov word ptr es:[ 4 * 4 + 0],isr4
mov word ptr es:[ 4 * 4 + 2],cs
mov al,127
mov bl,1
add al,bl
into ;if OF is set, generate int#4
.exit
isr4 proc
jmp l@1
msg: db "Overflow flag is set$"
l@1:
mov dx,offset msg
mov ah,9
int 0x21

iret
isr4 endp
```

Hook INT#10h

```
.model small
.data
.code
mov ax,@data
mov ds,ax

mov ax,0
mov es,ax

    ; Hook INT 0x10 to custom ISR
    mov word ptr es:[0x10 * 4 + 0], isr10    ; Offset of ISR
    mov word ptr es:[0x10 * 4 + 2], cs       ; Code segment of ISR

    ; Trigger INT 0x10 (BIOS for video mode setting)
    int 0x10

    ; End program
   .exit

isr10 proc
    jmp l@1
    msg: db "Custom int 0x10 is invoked!$"
    l@1:
    mov dx, offset msg
    mov ah, 9     ; BIOS interrupt 0x09 for output
    int 0x21      ; Print custom message

    iret          ; Return from interrupt
isr10 endp
```

Hook INT#21h

```
.model small
.data
.code
mov ax,@data
mov ds,ax

mov ax,0
mov es,ax

    ; Hook INT 0x21 to custom ISR
    mov word ptr es:[0x21*4+0], isr21     ; Offset of ISR
    mov word ptr es:[0x21*4+2], cs        ; Code segment of ISR

    ; Trigger INT 0x21 (normally handled by BIOS)
    int 0x21

    ; End program
   .exit

isr21 proc
    jmp l@1
    print_msg: db "Hello, Custom INT 0x21!$"
    l@1:
    mov ah, 0x09     ; BIOS interrupt 0x09 for output
    mov dx, offset print_msg
    int 0x21         ; Print custom message
    iret             ; Return from interrupt
isr21 endp
```

Practice Questions

1. **Hooking INT 0x10**

Task:

Hook INT 0x10 (video services) to display a custom message on the screen.

- **Service # Description**:

  0x0 - Clear screen

  0x2 - Print character at current cursor position

  0xE - Write string to screen

Write a program that hooks INT 0x10 to handle a custom message display.

Practice Questions

1. **Hook INT 0x70 for Addition**

Task:

Hook INT 0x70 with two services:

- **Service 0x1**: Input two 16-bit numbers from the user and store them in `AX` and `BX`.
- **Service 0x2**: Add the two 16-bit numbers (`AX + BX`) and display the result.